

We just need the *enode* details and have to replace the `[::]` with the localhost IP 127.0.0.1.

Let's fire up a new terminal, say Terminal 2, to host the second node. We proceed like, before and then supply the previous network ID, bootnode details and a port number for the node to bind to. We will also enable RPC from a particular domain, which we will use later to connect to this node.

```
$ cd ~/workspace/
$ mkdir node2
$ geth account new --datadir node2
$ geth init genesis.json --datadir node2
$ geth console --datadir node2 --networkid
1729 --port 30304 --bootnodes "enode://
c2b8714eca73d7a5a4264fa60641a8791ff8d33e47
dbb51f8b590594eb48e2aba9f360f340f358700e41e9d8415d7c
af70c67d12a66096053989c3824f7f64c3@[127.0.0.1]:30303" --rpc
--rpccorsdomain "http://remix.ethereum.org"
```

If everything goes well, you will have two nodes connected to each other, hosting the same blockchain. To confirm, you can use 'admin.peers' inside the Geth consoles.

Until now, we just hosted the blockchain. We did not perform any transaction, mine blocks or transfer ether. In fact, all our accounts are empty. Let us now check our account balance and start mining on one of the nodes, say Terminal 1.

```
> eth.getBalance(eth.accounts[0])
0
> miner.start(1)
```

We see that our account balance is zero. Starting the mining process may take some time. If you want a break, now is the time, because generating the DAG may take a considerable amount of time. Once the mining starts, you can recheck the account balance to see it increasing. This is because you are getting the mining reward. You can see logs on the console such as 'mined potential block' and 'commit new mining work' that tell you that a new block has been mined. The other node will also sync the newly created blocks.

To transfer ether from one account to another, we have to unlock the account and transfer funds. For example:

```
> personal.unlockAccount(eth.accounts[0], "pass", 24*7*360)
true
> eth.sendTransaction({from:eth.accounts[0],
to:"0xba993910008b3940626c83135fa2412b4a91b3b1", value: web3.
toWei(6, "ether")})
```

In the first command, the passphrase needs to be supplied in place of 'pass'. The last argument is the time duration for unlocking, which is optional. But we don't want this to bother us again and again.

In the second command, the 'to' field should contain the address of the account you want to send to. You can execute 'eth.accounts[0]' on Terminal 2 and supply the string here. Once done and once a new block is mined, you can see the change of funds reflected in your account balances. If it says insufficient funds, wait for some new blocks to be mined or decrease the amount. We have sent six ethers in the example shown.

Writing your first solidity contract

Now that we have set up the network to host the blockchain, let us deploy a smart contract and interact with it.

Fire up the browser and head to <http://remix.ethereum.org/>. We will use the online compiler to compile our code and with it, connect to our localhost and deploy on the private network. Close the default code (*Ballot.sol*) that is open on Remix, delete the *sol* file from under the browser dropdown on the left and create a new file named *Basic.sol* on the browser. Then write the following piece of code inside the contract.

```
pragma solidity ^0.4.19;

contract Basic {
    //declaring an unsigned interger. Let's make it private
    uint private variable;

    //this function returns the value of the stored variable
    function get() constant public returns (uint retVal) {
        return variable;
    }

    //this functions sets the value of the stored variable to
    the passed argument value
    function set (uint x) public {
        variable = x;
    }
}
```

Now, compile to see if it successfully compiles without errors. Next, go to the 'Run' tab and select 'Web3 Provider' from the 'Environment' drop down menu. Choose the default endpoint, if asked.

On clicking 'Create', the contract will be deployed. But on the browser console, you'll probably see an authentication error. This is because to deploy, your account needs to be unlocked. Head over to the Terminal 2 Geth console and unlock the account as you did while transferring funds. Now, once done, try creating the contract from your browser again. (If you see an error message saying 'Insufficient funds', try sending more ethers from your node1 account to your node2 account.) Once deployed and mined, you will see the address and function names on your browser, as seen on the lower right of Figure 1.